

Laboratório Aula 8 Reamostragem

September 7, 2026

1 Laboratório Avaliação em Cenários Críticos e Engenharia de Reamostragem

Curso de Aprendizado de Máquina

Objetivo: Compreender os desafios práticos do desbalanceamento de classes, implementar diagnósticos usando métricas de classificação assimétricas e dinâmicas, aplicar técnicas de reamostragem (SMOTE e NearMiss) de forma correta e evitar o vazamento de dados (*data leakage*).

1.0.1 Estrutura do Experimento:

1. Simulação do Dataset de Fraudes (Amostragem Populacional)
2. Partição Estratificada (Treino/Teste)
3. Modelo Baseline (Regressão Logística sem Tratamento)
4. Diagnóstico do Colapso (Matriz de Confusão, Acurácia, Precisão, Sensibilidade, F1, ROC/AUC e PR-AUC)
5. Intervenção via Reamostragem (SMOTE e NearMiss-1) unicamente em Treino
6. Análise Comparativa e Discussão dos Trade-offs de Engenharia

1.1 1. Configurando o Ambiente e Simulando os Dados desbalanceados

Para tornar este laboratório autossuficiente e reproduzível, geraremos um dataset sintético de transações com desbalanceamento extremo que simula perfeitamente o comportamento do clássico dataset `creditcard.csv` (onde a taxa de fraude é de aproximadamente **0.17%**).

```
[15]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score, fbeta_score,
    roc_curve, auc, precision_recall_curve
)
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import NearMiss
```

```
# Configurações de plotagem e semente de aleatoriedade
sns.set_theme(style="whitegrid")
np.random.seed(42)
```

```
[16]: X_raw, y_raw = make_classification(
    n_samples=50000,
    n_features=10,
    n_informative=8,
    n_redundant=2,
    weights=[0.9983, 0.0017], # 99.83% classe 0 (legítima), 0.17% classe 1
    ↪(fraude)
    n_clusters_per_class=1,
    random_state=42
)

# Criando um DataFrame pandas para facilitar a visualização
feature_cols = [f"V{i}" for i in range(1, 11)]
df = pd.DataFrame(X_raw, columns=feature_cols)
df['Class'] = y_raw

# Verificando a contagem absoluta e percentual de classes
class_counts = df['Class'].value_counts()
class_pcts = df['Class'].value_counts(normalize=True) * 100

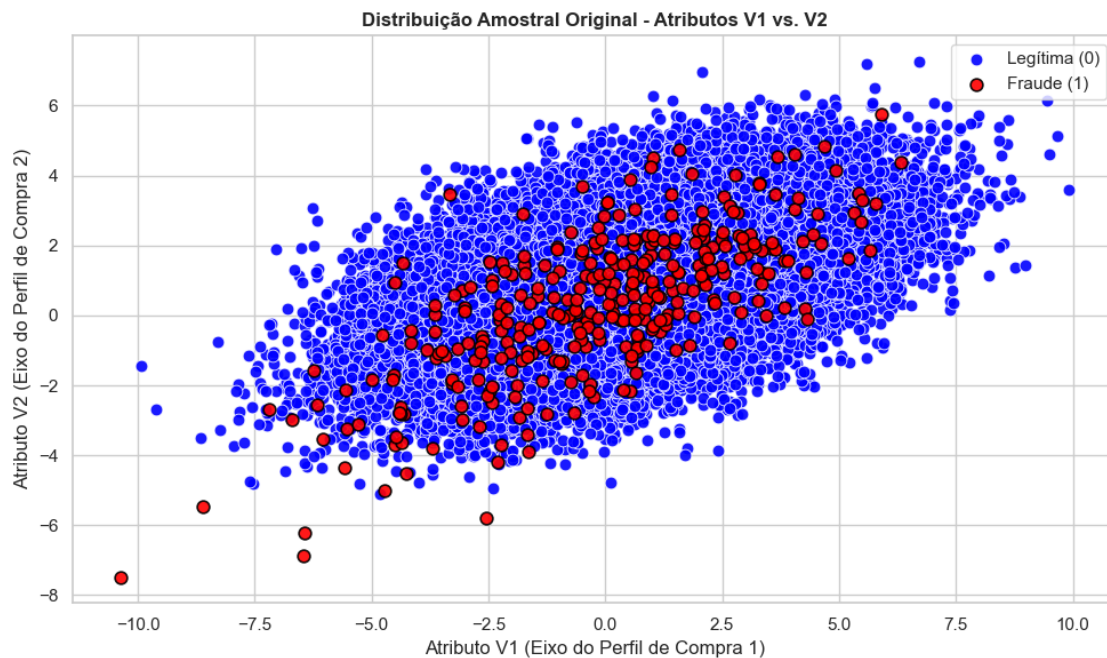
print("Distribuição Amostral Bruta:")
for cls, count in class_counts.items():
    print(f"Classe {cls}: {count:5d} transações ({class_pcts[cls]:.3f}%)")
```

```
Distribuição Amostral Bruta:
Classe 0: 49666 transações (99.332%)
Classe 1:   334 transações (0.668%)
```

```
[17]: plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df[df['Class'] == 0],
    x='V1', y='V2',
    color='blue', alpha=0.9, s=60, label='Legítima (0)'
)

sns.scatterplot(
    data=df[df['Class'] == 1],
    x='V1', y='V2',
    color='red', alpha=0.9, s=60, edgecolor='black', linewidth=1.2,
    ↪label='Fraude (1)', zorder=5
)
```

```
plt.title('Distribuição Amostral Original - Atributos V1 vs. V2', fontsize=12,
         fontweight='bold')
plt.xlabel('Atributo V1 (Eixo do Perfil de Compra 1)')
plt.ylabel('Atributo V2 (Eixo do Perfil de Compra 2)')
plt.legend(frameon=True, facecolor='white')
plt.tight_layout()
plt.show()
```



```
[18]: _, _, _, y_test_simple = train_test_split(
        X_raw, y_raw, test_size=0.3, random_state=10, stratify=None
    )

    X_train, X_test, y_train, y_test = train_test_split(
        X_raw, y_raw, test_size=0.3, random_state=42, stratify=y_raw
    )

    print(f"Proporção de Fraudes no Teste Simples: {np.mean(y_test_simple)*100:.4f}%")
    print(f"Proporção de Fraudes no Teste Estratificado: {np.mean(y_test)*100:.4f}%")
```

Proporção de Fraudes no Teste Simples: 0.6333%

Proporção de Fraudes no Teste Estratificado: 0.6667%

```
[19]: model_baseline = LogisticRegression(max_iter=1000, random_state=42)
    model_baseline.fit(X_train, y_train)
```

```
[19]: LogisticRegression(max_iter=1000, random_state=42)
```

```
[20]: y_pred_baseline = model_baseline.predict(X_test)
      y_prob_baseline = model_baseline.predict_proba(X_test)[:, 1]
```

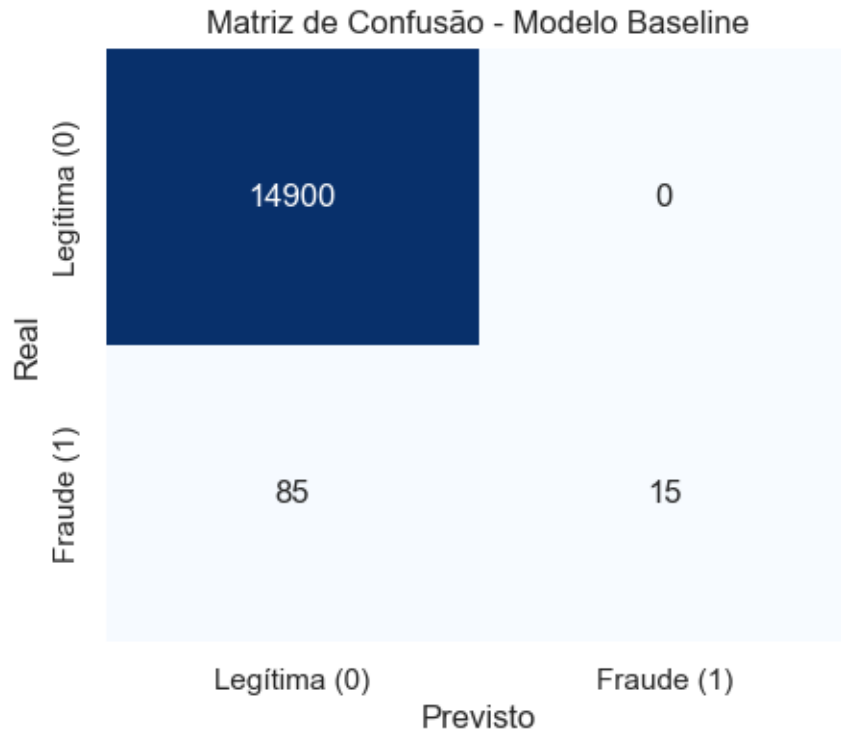
```
[21]: # 1. Acurácia Global
      acc_base = accuracy_score(y_test, y_pred_baseline)
      print(f"Acurácia Global do Baseline: {acc_base * 100:.4f}%")
```

Acurácia Global do Baseline: 99.4333%

```
[22]: cm_base = confusion_matrix(y_test, y_pred_baseline)
      tn, fp, fn, vp = cm_base.ravel()

      plt.figure(figsize=(5, 4))
      sns.heatmap(cm_base, annot=True, fmt="d", cmap="Blues", cbar=False,
                  xticklabels=["Legítima (0)", "Fraude (1)"],
                  yticklabels=["Legítima (0)", "Fraude (1)"])
      plt.title("Matriz de Confusão - Modelo Baseline")
      plt.xlabel("Previsto")
      plt.ylabel("Real")
      plt.show()

      print(f"Verdadeiros Negativos (VN): {tn}")
      print(f"Falsos Positivos (FP) - Alarme Falso: {fp}")
      print(f"Falsos Negativos (FN) - Fraudes Perdidas: {fn}")
      print(f"Verdadeiros Positivos (VP) - Fraudes Detectadas: {vp}")
```



Verdadeiros Negativos (VN): 14900

Falsos Positivos (FP) - Alarme Falso: 0

Falsos Negativos (FN) - Fraudes Perdidas: 85

Verdadeiros Positivos (VP) - Fraudes Detectadas: 15

```
[23]: prec_base = precision_score(y_test, y_pred_baseline, zero_division=0)
rec_base = recall_score(y_test, y_pred_baseline)
f1_base = f1_score(y_test, y_pred_baseline, zero_division=0)
f2_base = fbeta_score(y_test, y_pred_baseline, beta=2.0, zero_division=0)

print(f"Precisão: {prec_base * 100:.2f}% (Dos alarmes disparados, quantos eram
↪ fraudes reais)")
print(f"Sensibilidade: {rec_base * 100:.2f}% (Das fraudes reais existentes,
↪ quantas detectamos)")
print(f"F1-Score: {f1_base * 100:.2f}% (Média harmônica entre Precisão e
↪ Sensibilidade)")
print(f"F2-Score: {f2_base * 100:.2f}% (Dá duas vezes mais peso para a
↪ Sensibilidade)")
```

Precisão: 100.00% (Dos alarmes disparados, quantos eram fraudes reais)

Sensibilidade: 15.00% (Das fraudes reais existentes, quantas detectamos)

F1-Score: 26.09% (Média harmônica entre Precisão e Sensibilidade)

F2-Score: 18.07% (Dá duas vezes mais peso para a Sensibilidade)

1.1.1 Curva ROC vs. Curva Precisão-Sensibilidade (Efeito de Mascaramento)

Vamos desenhar as duas curvas lado a lado para visualizar como o enorme volume de Verdadeiros Negativos mascara a curva ROC.

```
[26]: fpr_base, tpr_base, _ = roc_curve(y_test, y_prob_baseline)
      roc_auc_base = auc(fpr_base, tpr_base)

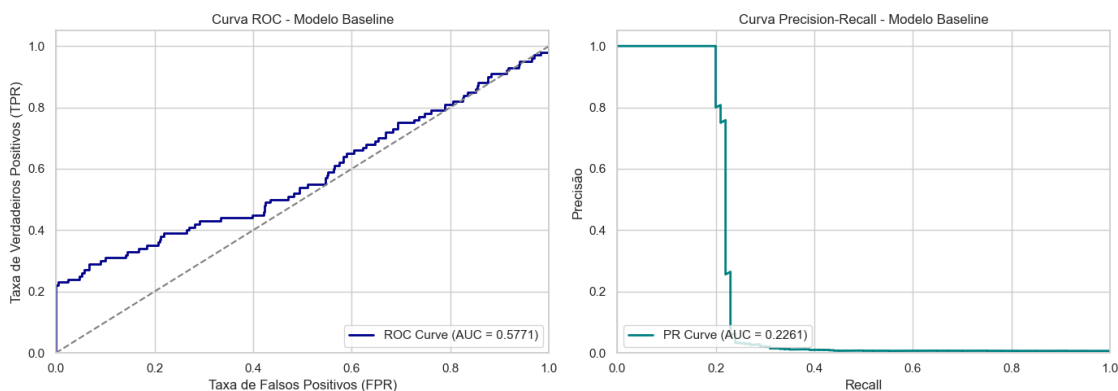
      precision_base, recall_base, _ = precision_recall_curve(y_test, y_prob_baseline)
      pr_auc_base = auc(recall_base, precision_base)

      fig, ax = plt.subplots(1, 2, figsize=(14, 5))

      ax[0].plot(fpr_base, tpr_base, color='darkblue', lw=2, label=f'ROC Curve (AUC = {roc_auc_base:.4f})')
      ax[0].plot([0, 1], [0, 1], color='gray', linestyle='--')
      ax[0].set_xlim([0.0, 1.0])
      ax[0].set_ylim([0.0, 1.05])
      ax[0].set_xlabel('Taxa de Falsos Positivos (FPR)')
      ax[0].set_ylabel('Taxa de Verdadeiros Positivos (TPR)')
      ax[0].set_title('Curva ROC - Modelo Baseline')
      ax[0].legend(loc="lower right")

      ax[1].plot(recall_base, precision_base, color='teal', lw=2, label=f'PR Curve (AUC = {pr_auc_base:.4f})')
      ax[1].set_xlim([0.0, 1.0])
      ax[1].set_ylim([0.0, 1.05])
      ax[1].set_xlabel('Recall')
      ax[1].set_ylabel('Precisão')
      ax[1].set_title('Curva Precision-Recall - Modelo Baseline')
      ax[1].legend(loc="lower left")

      plt.tight_layout()
      plt.show()
```



$$FPR = FP / (VN + FP)$$

```
[27]: smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

nearmiss = NearMiss(version=1)
X_train_nm, y_train_nm = nearmiss.fit_resample(X_train, y_train)
```

```
[28]: print("Equilíbrio de Classes Original no Treino:")
print(f" Classe 0: {np.sum(y_train == 0):5d} | Classe 1: {np.sum(y_train == 1):5d}")

print("\nEquilíbrio de Classes pós-SMOTE no Treino:")
print(f" Classe 0: {np.sum(y_train_smote == 0):5d} | Classe 1: {np.sum(y_train_smote == 1):5d}")

print("\nEquilíbrio de Classes pós-NearMiss-1 no Treino:")
print(f" Classe 0: {np.sum(y_train_nm == 0):5d} | Classe 1: {np.sum(y_train_nm == 1):5d}")
```

Equilíbrio de Classes Original no Treino:

Classe 0: 34766 | Classe 1: 234

Equilíbrio de Classes pós-SMOTE no Treino:

Classe 0: 34766 | Classe 1: 34766

Equilíbrio de Classes pós-NearMiss-1 no Treino:

Classe 0: 234 | Classe 1: 234

```
[29]: # Plot do impacto das reamostragens no treino
fig, ax = plt.subplots(1, 2, figsize=(16, 6))

# 1. Plot do Espaço de Treino pós-SMOTE
df_train_smote = pd.DataFrame(X_train_smote, columns=feature_cols)
df_train_smote['Class'] = y_train_smote

sns.scatterplot(
    data=df_train_smote[df_train_smote['Class'] == 0],
    x='V1', y='V2',
    color='skyblue', alpha=0.15, s=10, label='Legítima Original (0)', ax=ax[0]
)
sns.scatterplot(
    data=df_train_smote[df_train_smote['Class'] == 1],
    x='V1', y='V2',
    color='salmon', alpha=0.5, s=15, label='Sintética pós-SMOTE (1)', ax=ax[0]
)
# Destaque para as fraudes originais de treino
ax[0].scatter(
```

```

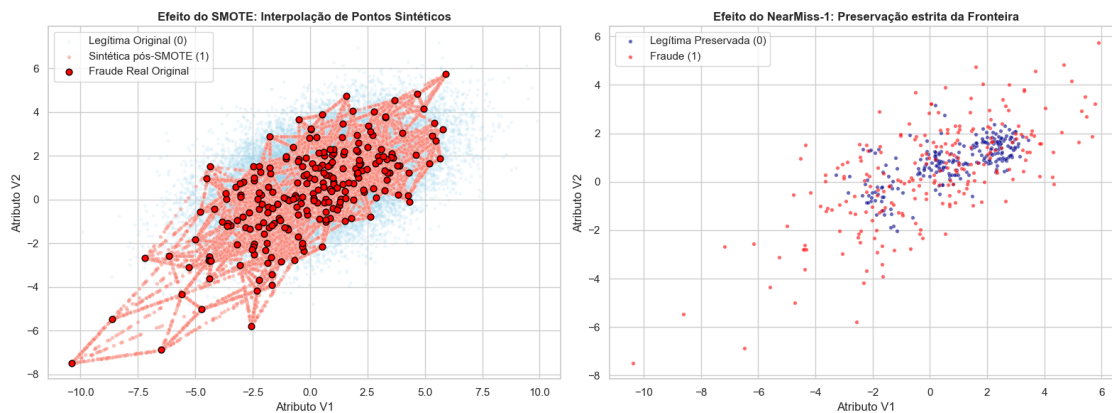
X_train[y_train == 1][:, 0], X_train[y_train == 1][:, 1],
    color='red', s=40, edgecolor='black', label='Fraude Real Original', zorder=5
)
ax[0].set_title('Efeito do SMOTE: Interpolação de Pontos Sintéticos',
    ↪fontsize=12, fontweight='bold')
ax[0].set_xlabel('Atributo V1')
ax[0].set_ylabel('Atributo V2')
ax[0].legend(frameon=True, facecolor='white')

# 2. Plot do Espaço de Treino pós-NearMiss-1
df_train_nm = pd.DataFrame(X_train_nm, columns=feature_cols)
df_train_nm['Class'] = y_train_nm

sns.scatterplot(
    data=df_train_nm[df_train_nm['Class'] == 0],
    x='V1', y='V2',
    color='darkblue', alpha=0.6, s=15, label='Legítima Preservada (0)', ax=ax[1]
)
sns.scatterplot(
    data=df_train_nm[df_train_nm['Class'] == 1],
    x='V1', y='V2',
    color='red', alpha=0.6, s=15, label='Fraude (1)', ax=ax[1]
)
ax[1].set_title('Efeito do NearMiss-1: Preservação estrita da Fronteira',
    ↪fontsize=12, fontweight='bold')
ax[1].set_xlabel('Atributo V1')
ax[1].set_ylabel('Atributo V2')
ax[1].legend(frameon=True, facecolor='white')

plt.tight_layout()
plt.show()

```



```
[30]: # Ajuste com SMOTE
model_smote = LogisticRegression(max_iter=1000, random_state=42)
model_smote.fit(X_train_smote, y_train_smote)
y_pred_smote = model_smote.predict(X_test)
y_prob_smote = model_smote.predict_proba(X_test)[:, 1]

# Ajuste com NearMiss-1
model_nm = LogisticRegression(max_iter=1000, random_state=42)
model_nm.fit(X_train_nm, y_train_nm)
y_pred_nm = model_nm.predict(X_test)
y_prob_nm = model_nm.predict_proba(X_test)[:, 1]

[31]: metrics_list = []

for name, y_p, y_pr in [
    ("Baseline (Desbalanceado)", y_pred_baseline, y_prob_baseline),
    ("Tratado com SMOTE", y_pred_smote, y_prob_smote),
    ("Tratado com NearMiss-1", y_pred_nm, y_prob_nm)
]:
    cm = confusion_matrix(y_test, y_p)
    tn, fp, fn, vp = cm.ravel()
    fpr_vec, tpr_vec, _ = roc_curve(y_test, y_pr)
    p_vec, r_vec, _ = precision_recall_curve(y_test, y_pr)

    metrics_list.append({
        "Abordagem": name,
        "Acurácia (%)": accuracy_score(y_test, y_p) * 100,
        "Precisão (%)": precision_score(y_test, y_p, zero_division=0) * 100,
        "Recall (%)": recall_score(y_test, y_p) * 100,
        "F1-Score (%)": f1_score(y_test, y_p, zero_division=0) * 100,
        "F2-Score (%)": fbeta_score(y_test, y_p, beta=2.0, zero_division=0) * 100,
        "AUC-ROC": auc(fpr_vec, tpr_vec),
        "AUC-PR": auc(r_vec, p_vec),
        "Falsos Positivos (FP)": fp,
        "Falsos Negativos (FN)": fn
    })

df_metrics = pd.DataFrame(metrics_list)
display(df_metrics.round(4))
```

	Abordagem	Acurácia (%)	Precisão (%)	Recall (%)	\
0	Baseline (Desbalanceado)	99.4333	100.0000	15.0	
1	Tratado com SMOTE	71.9533	0.9788	41.0	
2	Tratado com NearMiss-1	57.5733	0.8783	56.0	

	F1-Score (%)	F2-Score (%)	AUC-ROC	AUC-PR	Falsos Positivos (FP)	\
0	26.0870	18.0723	0.5771	0.2261	0	

1	1.9119	4.4672	0.5843	0.2247	4148
2	1.7295	4.1322	0.6045	0.1691	6320

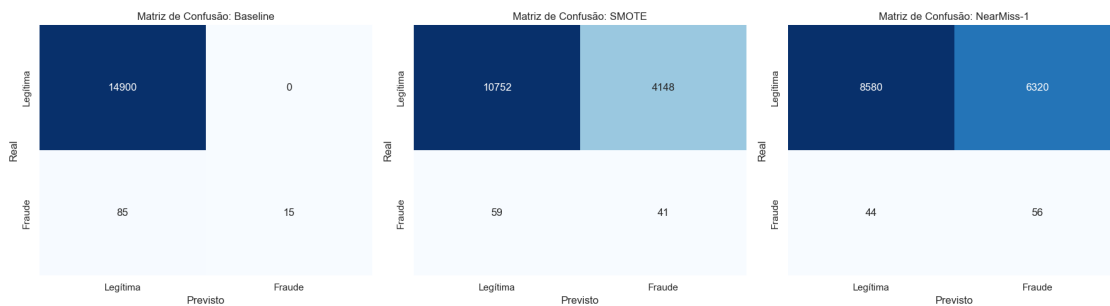
Falsos Negativos (FN)

0	85
1	59
2	44

```
[32]: fig, ax = plt.subplots(1, 3, figsize=(18, 5))

for idx, (name, y_p) in enumerate([
    ("Baseline", y_pred_baseline),
    ("SMOTE", y_pred_smote),
    ("NearMiss-1", y_pred_nm)
]):
    cm = confusion_matrix(y_test, y_p)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, ax=ax[idx],
                xticklabels=["Legítima", "Fraude"], yticklabels=["Legítima",
    ↪ "Fraude"])
    ax[idx].set_title(f"Matriz de Confusão: {name}")
    ax[idx].set_xlabel("Previsto")
    ax[idx].set_ylabel("Real")

plt.tight_layout()
plt.show()
```



```
[33]: fig, ax = plt.subplots(1, 2, figsize=(16, 6))
colors = ['darkblue', 'green', 'orange']
labels = ["Baseline", "SMOTE", "NearMiss-1"]
probs = [y_prob_baseline, y_prob_smote, y_prob_nm]

for color, label, prob in zip(colors, labels, probs):
    # ROC Curve
    fpr, tpr, _ = roc_curve(y_test, prob)
    ax[0].plot(fpr, tpr, color=color, lw=2, label=f'{label} (AUC = {auc(fpr,
    ↪ tpr):.3f})')
    ax[1].plot(fpr, tpr, color='black', lw=1, label='Random')
```

```

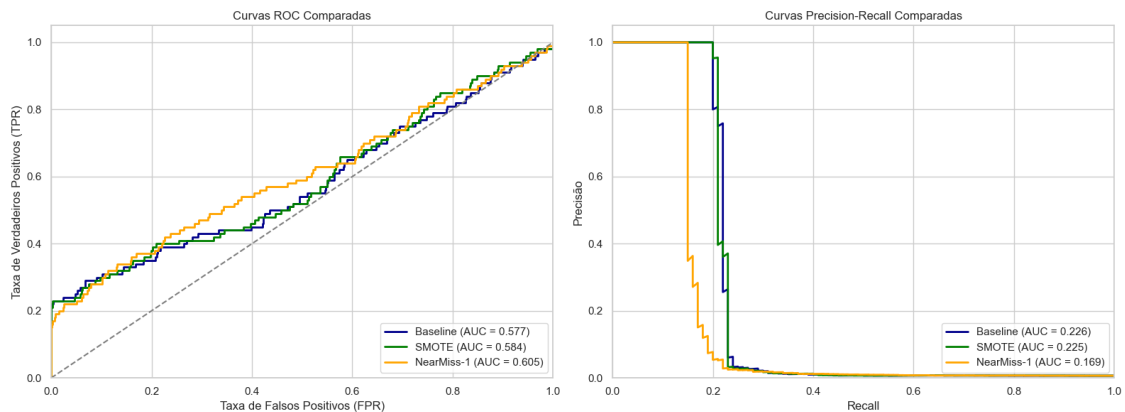
# PR Curve
p, r, _ = precision_recall_curve(y_test, prob)
ax[1].plot(r, p, color=color, lw=2, label=f'{label} (AUC = {auc(r, p):.
↪3f})')

ax[0].plot([0, 1], [0, 1], color='gray', linestyle='--')
ax[0].set_xlim([0.0, 1.0])
ax[0].set_ylim([0.0, 1.05])
ax[0].set_xlabel('Taxa de Falsos Positivos (FPR)')
ax[0].set_ylabel('Taxa de Verdadeiros Positivos (TPR)')
ax[0].set_title('Curvas ROC Comparadas')
ax[0].legend(loc="lower right")

ax[1].set_xlim([0.0, 1.0])
ax[1].set_ylim([0.0, 1.05])
ax[1].set_xlabel('Recall')
ax[1].set_ylabel('Precisão')
ax[1].set_title('Curvas Precision-Recall Comparadas')
ax[1].legend(loc="lower right")

plt.tight_layout()
plt.show()

```



[]: